Research Foundations for ContextKeeper WP-003

This report synthesizes published evidence across eight technical domains underpinning the ContextKeeper Platform Architecture and Token Intelligence white paper. The findings validate several core claims—diff-based context injection ratios are consistent with published compression research, and prompt caching yields 60–90% cost savings in engineering workflows—while identifying genuine research gaps in context exhaustion prediction, cross-model state transfer, and dependency-aware prioritization. Each section provides specific citations, quantitative findings, and source-type tags for direct integration into WP-003.

Topic 1: Context windows grow cheaper but not smarter

The "lost in the middle" problem persists through 2025–2026

The foundational finding comes from Liu et al. (2024), "Lost in the Middle: How Language Models Use Long Contexts," published in *Transactions of the Association for Computational Linguistics*, 12:157–173 (arXiv) (arXiv:2307.03172) [peer-reviewed]. Performance follows a **U-shaped curve**: models perform best when relevant information sits at the beginning or end of the context, degrading significantly when key information is in the middle. (arXiv) (MIT Press) This finding has been reinforced, not resolved, by subsequent work. (Diffray)

The RULER benchmark (Hsieh et al., 2024, COLM 2024, arXiv:2404.06654) [peer-reviewed] found that despite near-perfect Needle-in-a-Haystack scores, **only half of models claiming 32K+ context maintained satisfactory performance** on complex tasks at that length. (arXiv) GPT-4 showed the least degradation but still dropped **15.4 points at 128K tokens**. (arXiv) (Medium) The NoLiMa benchmark (Modarressi et al., 2025, ICML 2025, arXiv:2502.05167) [peer-reviewed] tested non-literal matching and found results even more damning: at 32K tokens, **10 of 12 models dropped below 50% of short-context baseline performance**, (16x Eval) with GPT-4o falling from **99.3% to 69.7%**. Most models' effective length was ≤2K tokens versus claimed 128K+.

Context rot is real and universal

The term "context rot" was formalized by Hong, Troynikov, and Huber (2025) at Chroma, who tested **18 frontier LLMs** including GPT-4.1, Claude 4, Gemini 2.5, and Qwen3 (trychroma) [practitioner/community evidence]. **All 18 models showed degradation as input length increased**, even on simple tasks. (Diffray) Degradation is continuous, not a cliff—it occurs well before context window limits. (Morph) A critical finding: performance degradation is independent of content quality. Du et al. (2025), "Context Length Alone Hurts LLM Performance Despite Perfect Retrieval" (Findings of EMNLP 2025, arXiv:2510.05381) [peer-reviewed] demonstrated that even with **100% perfect retrieval**, performance degrades **13.9–85%** as input length

increases. Even masking all irrelevant tokens doesn't eliminate the effect. Sheer context length imposes a "cognitive tax." (arXiv) (Cool Papers) Multi-turn sensitivity was confirmed by arXiv:2506.00069 (2025) [preprint], showing Gemini Pro accuracy drops **5.7%** from zero-context to ~4K–16K tokens. (arXiv)

Anthropic's own engineering blog (2025) characterizes context as a **"finite resource with diminishing marginal returns"** (Anthropic) [vendor documentation], noting that attention is quadratic—at 100K tokens, the model must manage **10 billion pairwise relationships**. (Medium +2) Context pollution, not model capability, is the primary failure mode for coding agents. (Letta) (Codeaholicguy)

RAG remains ~100× cheaper than long context

Li et al. (2024), "Retrieval Augmented Generation or Long-Context LLMs?" (EMNLP 2024 Industry Track, arXiv:2407.16833) [peer-reviewed] found that when sufficiently resourced, **long context consistently outperforms RAG** in average performance, but **RAG's significantly lower cost** remains a distinct advantage. (arXiv) (arxiv) Their Self-Route hybrid approach optimizes both. (arXiv) Practitioner data shows full-context queries of a book via Claude cost approximately **$3 per query** versus **$0.03 via RAG**—a 100× difference (Legion Secure AI) [practitioner/community evidence]. The LaRA benchmark (Li et al., 2025) [preprint] confirms no silver bullet: optimal strategy depends on model size, task type, context length, and retrieval quality. (Meilisearch)

Token pricing has fallen 1,000× for 2023-era capabilities

Epoch AI's analysis (Cottier et al., 2025) [practitioner/community evidence] found the price to achieve GPT-4's performance fell by **40× per year**, with the median rate accelerating to **200× per year** after January 2024. (Epoch AI) (epoch) Fradkin et al. (2025) [preprint] documented that models state-of-the-art in 2023 have experienced ~1,000× price declines, but average price paid per token has remained relatively constant—demand shifts to superior intelligence. (Andreyfradkin) A critical caveat: reasoning models (o1, o3, DeepSeek-R1) generate thousands of internal tokens, actually increasing total costs despite lower per-token prices (IKANGAI) (WorkOS) [practitioner/community evidence].

Token consumption in engineering workflows

The OpenRouter State of AI Report (2025) [vendor documentation], analyzing **100+ trillion tokens**, found average prompt length has grown **nearly fourfold** since early 2024. (OpenRouter) Practitioner estimates for Cursor Pro put typical usage at **~1.3M tokens/month**, with power users substantially exceeding this [practitioner/community evidence]. (Substack) GitHub Copilot now supports up to **192K-token context** with Codex-1, (Quantumrun) with automatic compaction at 95% capacity (DeepWiki) [vendor documentation]. One developer's Claude Code usage over one month cost approximately **$480** (Couch, 2026) [practitioner/community evidence]. (Simon P. Couch)

Research gap: No published longitudinal study tracks token consumption growth curves across different workflow types.

Topic 2: Diff-based injection claims are plausible but not directly validated

The 85–95% claim aligns with published compression research

No single paper directly measures diff-based context injection at 85–95% reduction for AI coding agents. However, multiple converging lines of evidence support the range as consistent with measured compression ratios.

The strongest direct evidence comes from Lewis (2026), "Structured Distillation for Personalized Agent Memory" (arXiv:2603.13017) [preprint], which demonstrated **11× compression (~91% reduction)** of agent conversation history—from 371 tokens per exchange to 38 tokens via structured distillation—while preserving retrieval quality (arXiv) across 4,182 conversations (arXiv) and 214,519 consensus-graded query-result pairs. The LLMLingua family from Microsoft Research provides the peer-reviewed backbone: the original LLMLingua (EMNLP 2023) [peer-reviewed] achieved **up to 20× compression (95% reduction)** with only (arXiv) (OpenReview) **1.5% performance loss** on GSM8K reasoning. (Llmlingua) LongLLMLingua (ACL 2024) [peer-reviewed] achieved **94% cost reduction** on the LooGLE benchmark, (arXiv) (ACL Anthology) and notably found that compression can *improve* performance by increasing information density and mitigating lost-in-the-middle effects. Selective Context (Li et al., EMNLP 2023) [peer-reviewed] achieved ~50% compression using self-information filtering. (GitHub) (arXiv)

The JetBrains study (Lindenbauer et al., NeurIPS 2025 DL4C Workshop, arXiv:2508.21433) [workshop paper] found that simple observation masking—hiding verbose past tool outputs—halves cost while matching LLM summarization performance for SE agents. (GitHub) (JetBrains) The key finding from Lewis (2026) explains why high compression is achievable: "A large fraction of those tokens is irrelevant to retrieval: tool output, boilerplate responses, tangential discussion, repeated file contents." (arXiv)

McMillan 2026: file-native context engineering

McMillan (2026), "Structured Context Engineering for File-Native Agentic Systems" (arXiv:2602.05447v2) [preprint] ran 9,649 experiments across 11 models and 4 formats. For frontier models, file-based retrieval showed **+2.7% accuracy improvement** ($p=0.029$) over prompt injection. However, open-source models showed **-7.7% aggregate** ($p<0.001$). Format (YAML, Markdown, JSON, TOON) did not significantly affect aggregate accuracy (chi-squared=2.45, $p=0.484$). (arXiv) (alphaXiv) This validates selective retrieval over full injection but reveals the benefit is model-dependent.

What LLMs actually need from prior sessions

Lewis (2026) identified that only four fields survive 11× compression without retrieval degradation: exchange_core, specific_context, thematic assignments, and files_touched. The Lore protocol (arXiv:2603.15566, 2026) [preprint] emphasizes that decisions made, rejected alternatives, constraints, and confidence assessments matter for cross-session continuity—and these exist only in conversation, not in diffs. (arXiv) SWE-agent ablation studies (arXiv:2405.15793, 2024) [peer-reviewed] found that keeping only the latest ~100 lines of file context outperforms full-file display, (arXiv) and the Verdent SWE-bench report (2026) found a minimal toolset (bash, read, write, edit) sufficient for benchmark success. (Verdent)

Recommended citation framing for WP-003: "Published research demonstrates 4×–20× compression ratios (75–95% reduction) with minimal performance loss [LLMLingua, EMNLP 2023; LongLLMLingua, ACL 2024], and structured distillation of agent conversation history achieves 11× compression (~91% reduction) while preserving retrieval quality [Lewis, 2026]."

Research gap: No published controlled experiment specifically compares diff-only injection versus full-history injection in an AI coding agent workflow with measured token reduction ratios and task performance metrics.

Topic 3: TF-IDF is demonstrably inferior to BM25 and hybrid retrieval

Sparse retrieval generalizes better than dense in zero-shot settings

Zeng, Zamani, and Vinay (2025), "Scaling Sparse and Dense Retrieval in Decoder-Only LLMs" (SIGIR 2025, arXiv:2502.15526) [peer-reviewed] found that sparse retrieval consistently outperforms dense retrieval across all evaluation benchmarks, with sparse-KD surpassing dense-KD by **10.5% on BEIR** (out-of-domain). The BEIR benchmark (Thakur et al., NeurIPS 2021) [peer-reviewed] established BM25 at avg nDCG@10 of ~43.4—dense retrieval models (DPR) underperformed BM25 on 10+ of 18 out-of-domain datasets. Hybrid retrieval improves this to **52.6 nDCG@10**, (Emergent Mind) a ~21% relative gain (Abdallah et al., 2025, arXiv:2502.20245) [preprint].

TF-IDF's weakness on short documents is well-documented

TF-IDF's effectiveness specifically degrades on short documents because term frequency statistics become sparse—most terms appear only once, making TF values non-discriminative while IDF scores become disproportionately influential. BM25 addresses this with term frequency saturation (k1 parameter) and **document length normalization** (b parameter), making it strictly superior for mixed-length corpora [established IR theory, Robertson & Walker, 1994]. For agent memory stores containing short structured entries (task summaries, memory notes), this weakness is amplified.

The industry has converged on hybrid retrieval

All major agent memory systems now use hybrid retrieval as the underlying mechanism. MemGPT/Letta (Packer et al., arXiv:2310.08560) (arXiv) [preprint, now productionized] achieves ~83.2% on the LoCoMo benchmark using embedding-based archival retrieval. (arXiv) Mem0 (Chhikara et al., 2025, arXiv:2504.19413) [preprint] reports **26% relative improvement** over OpenAI baselines with **91% lower p95 latency** and **90%+ token cost savings**, using just 7K tokens per conversation versus 26K+ for full context. Zep (Rasmussen et al., 2025, arXiv:2501.13956) [preprint] adds temporal knowledge graphs with hybrid BGE-m3 embeddings and keyword search.

Practitioner benchmarks show hybrid search (dense + SPLADE sparse) improved MRR by **18.5%** over dense-only in real-world testing, but only when fusion parameters are tuned—poorly-tuned hybrid performed *worse* than dense-only [practitioner/community evidence]. (AIMultiple) Reciprocal Rank Fusion (RRF) with k=60 (Medium) is the standard fusion technique, requiring no tuning.

LangChain uses EnsembleRetriever with configurable weights (e.g., 0.4 BM25 / 0.6 dense); (Medium) LlamaIndex uses QueryFusionRetriever; (Llamaindex) Elasticsearch offers native hybrid retrieval [vendor documentation].

Research gap: No published work benchmarks retrieval methods specifically on short agent memory entries (task summaries, structured notes). Most IR benchmarks focus on passage-level or document-level retrieval. The behavior of TF-IDF/BM25 versus dense retrieval as an agent's memory grows from 100 to 10,000+ entries is unstudied.

Topic 4: DAG persistence is well-supported by frameworks but under-studied academically

Academic DAG planning work is growing rapidly

DynTaskMAS (AAAI ICAPS 2025, arXiv:2503.07675) [peer-reviewed] introduces a Dynamic Task Graph Generator constructing DAGs with interdependencies and an Asynchronous Parallel Execution Engine, achieving **21–33% execution time improvements** and **3.47× throughput** for 16 concurrent agents. (arXiv) DeMAC (EMNLP 2025 Findings) [peer-reviewed] uses dynamic DAG updates in multi-agent coordination. The survey by Xu Huang et al. (2024, arXiv:2402.02716) [preprint] provides the first systematic taxonomy of LLM-agent planning: Task Decomposition, Plan Selection, External Module, Reflection, and Memory-augmented Planning. (Lims) (arXiv)

Framework comparison reveals divergent persistence strategies

- **LangGraph**: The most mature persistence model. Built-in **Checkpointers** save graph state snapshots at every super-step, organized into threads. Langchain Supports SQLite, PostgreSQL, DynamoDB backends. Pending writes are preserved on failure; resume doesn't re-run successful nodes. Supports time-travel debugging and human-in-the-loop interrupts AWS [vendor documentation].
- **CrewAI**: Offers `@persist` decorator for automatic state persistence with SQLiteFlowPersistence. Event-driven architecture with `@start()` and `@listen()` decorators. Default in-memory execution is "a non-starter for serious applications" per practitioner feedback [vendor documentation].
- **AutoGen/Microsoft Agent Framework**: AutoGen's GraphFlow **does not provide built-in checkpointing**; GitHub Issue #7043 documents state corruption bugs during agent transitions. The successor Microsoft Agent Framework (public preview Oct 2025, GA target Q1 2026) adds automatic checkpointing via `FileCheckpointStorage` with granular recovery [vendor documentation].
- **Temporal.io**: The gold standard for durable execution—workflows automatically capture state at every step via event history. Workflows can run for days/months/years without state loss. Temporal Production users include **NVIDIA, OpenAI Codex, and Replit Agent**. $5B valuation as of 2025 [vendor documentation].
- **Letta/MemGPT**: Model-agnostic agent serialization via the open Agent File (.af) format, designed to carry memories across model generations [vendor documentation].

State machines for agent task management

StateFlow (arXiv:2403.11322, 2024) [preprint] formally models LLM workflows as transducer FSMs defined as a sextuple $\langle \Sigma, \Gamma, S, s_0, \delta, \omega \rangle$. MetaAgent (ICML 2025) [peer-reviewed] extends this with automatic FSM-based multi-agent system generation, including LLM-based state merging optimization. The Google A2A protocol defines task status as a well-defined FSM: `submitted → working → input-required / completed / failed / canceled` [preprint].

Research gaps flagged:

- No large-scale controlled study isolates the impact of plan persistence on task completion rates. The closest is WebArena Experience Replay (Liu et al., 2025): contextual experience replay raised GPT-4o success rates by **51% relative improvement** [preprint], but conflates memory with other architectural changes.
- **Plan portability across model boundaries is a genuine gap.** No peer-reviewed work addresses transferring a partially-executed plan between different LLM providers. Letta's

Agent File (.af) is the closest attempt at a portable format.

- **DEL (Dynamic Epistemic Logic) has not been applied to LLM agent task management.** The formal methods and agent engineering communities remain disconnected here.

Topic 5: Linear regression is suboptimal for context exhaustion prediction

Token consumption is non-linear and highly variable in agentic workflows

The most directly relevant empirical study is Bai et al. (2025), "How Do Coding Agents Spend Your Money?" (ICLR 2026 submission, withdrawn, OpenReview #20681) [preprint—treat with caution]. openreview Key finding: token usage exhibits **large variance across runs—up to 10× for identical tasks**. Input tokens dominate consumption in agentic workflows. OpenReview Most critically, predicting total token consumption from task descriptions achieves only **Pearson's r < 0.15**, indicating the prediction problem may be fundamentally intractable from initial conditions.

Dibia's practitioner measurements show non-linear jumps in naive agents: 888 → 3,400 → 8,900 → 14,200 tokens across four iterations, driven by tool results (file reads, directory listings) [practitioner/community evidence]. Victordibia Anthropic's documentation confirms simple multi-turn conversations grow linearly, Claude API Docs but this breaks down immediately for agentic workflows JetBrains [vendor documentation].

Reasoning tokens are fundamentally unpredictable

Research on reasoning token consumption reveals high variability and poor correlation with output quality. "Think Deep, Not Just Long" (arXiv:2602.13517, 2026) [preprint] found a **negative correlation (r = -0.59)** between reasoning length and performance—longer reasoning often means *worse* results. arXiv "Not All Tokens Are What You Need In Thinking" (arXiv:2505.17827) [preprint] demonstrated that arXiv from 54M reasoning tokens, only **8M (~15%) were essential**. DeepSeek-R1 generates an average of 1,388 thinking tokens per sample, but these can be reduced by **nearly half** without performance loss Mcgill-nlp (McGill NLP, 2025) [preprint]. The Reasoning Completion Point (RCP) method (arXiv:2508.17627) [preprint] reduces token usage by **up to 44%** while preserving accuracy, confirming that reasoning consumption has instance-specific completion points that cannot be predicted from trailing patterns. arXiv

Better alternatives exist

For non-stationary, bursty time series, established alternatives include:

- **Holt's exponential smoothing with damped trend**, recommended for "situations where the future is very uncertain" Duke University (Hyndman & Athanasopoulos, *Forecasting: Principles and Practice*) [textbook]

- **Bayesian Online Changepoint Detection** (Adams & MacKay, 2007) [peer-reviewed], which maintains run-length distributions and detects regime changes in streaming data
- **TCDformer** (2024, *Journal of King Saud University*) [peer-reviewed], combining change-point detection with prediction, reducing MSE by **47.36%** versus standard methods on non-stationary data (ScienceDirect)

The industry has converged on **threshold-based triggers** rather than prediction: Goose (Block/Square) uses 80% of token limit; (Goose) Factory.ai uses dual thresholds (fill line / drain line); (Factory) ForgeCode uses triple thresholds (token, message, turn) (ForgeCode) [vendor documentation]. This convergence suggests practitioners have implicitly concluded that prediction isn't worth the complexity given high variance.

Token budget estimation per-request shows more promise: Han et al. (2024), "Token-Budget-Aware LLM Reasoning" (ACL 2025, arXiv:2412.18547) [peer-reviewed] reduces output token costs by **68.64%** with minimal accuracy loss. (arXiv) SelfBudgeter (arXiv:2505.11274) [preprint] achieves **74.47% reduction in response length** on MATH while maintaining accuracy. (arXiv)

Research gaps: No published work applies time-series forecasting to predict context window exhaustion from trailing consumption data. No empirical characterization of consumption curves across workflow types. No published "context health monitoring" framework. The Bai et al. paper was withdrawn from ICLR 2026, leaving this area without peer-reviewed work.

Topic 6: Model routing saves 50–98% but mid-task switching degrades quality by ±4 F1

The routing literature is robust and peer-reviewed

FrugalGPT (Chen, Zaharia, Zou; *TMLR* 2024, arXiv:2305.05176) [peer-reviewed] demonstrated **up to 98% cost reduction** while matching GPT-4 performance, or **+4% accuracy over GPT-4** at the same cost. (arXiv) (arXiv) The approach uses LLM cascades—sequentially querying from cheapest to most expensive, stopping when a scoring function rates the response as reliable. (arXiv) (Lingjiaochen) Updated with modern models including Claude 3.5, Gemini 1.5, and GPT-4o. (Lingjiaochen) ~160 citations.

RouteLLM (Ong, Almahairi et al., ICLR 2025, arXiv:2406.18665) [peer-reviewed] trains lightweight routers using Chatbot Arena preference data, achieving **>2× cost reduction** without quality sacrifice. (Iclr) The matrix factorization router achieved **95% of GPT-4 performance with only 26% of GPT-4 calls**. Critically, routers trained on one model pair transfer well to others without retraining. (OpenReview) (Iclr)

Additional peer-reviewed routing work includes **Hybrid LLM** (Ding et al., ICLR 2024) achieving **40% fewer calls** to the large model with no quality drop [peer-reviewed]; (OpenReview) **BEST-**

Route (Ding et al., ICML 2025) reducing costs by **up to 60% with <1% performance drop** [peer-reviewed]; GitHub and **Cascade Routing** (Dekoninck et al., ICML 2025) achieving **5× or more cost savings** at zero degradation when the quality judge reliability exceeds ~80% OpenReview [peer-reviewed]. arXiv

Mid-task model switching introduces measurable drift

The key paper is Khraishi et al. (2025), "Evaluating Performance Drift from Model Switching in Multi-Turn LLM Systems" (arXiv:2603.03111, NatWest AI Research/UCL) [preprint]. Even a **single-turn handoff** produces statistically significant drift of **±4 absolute F1 on CoQA**—comparable to the performance gap between model tiers. arXiv **Within-family switching** (e.g., GPT-5 variants) yields near-zero deltas, but **cross-family switching** shows substantial degradation. arXiv Models inherit "conversational state" (entity choices, coreference resolutions) from prior assistant turns rather than fully re-grounding on source material. arXiv LLM stability research (arXiv:2408.04667v2) [preprint] found that even identical single LLMs show accuracy variations up to **10%** across repeated runs, with multiplicative degradation in multi-step systems: four classifiers at 95% stability yield **0.95^4 = 0.814 expected performance**. arXiv

Critical caveat for WP-003: Cost savings from routing papers assume per-query independence. They do not address multi-turn stateful conversations where switching models mid-task introduces drift. arXiv The gap between single-query routing savings and multi-turn stateful deployment remains largely unstudied.

Research gaps: No standardized benchmark for measuring state transfer fidelity between models. No published "model-agnostic state representation" format. No systematic comparison of information loss when migrating conversations between specific model pairs beyond Khraishi et al.

Topic 7: Prompt caching yields 60–92% savings when architecturally sound

Provider pricing comparison

Feature	Anthropic Claude	OpenAI	Google Gemini
Cache read discount	**90%** (0.1× base)	50% (0.5× base)	75–90% (model-dependent)
Cache write premium	25% (5-min TTL) / 100% (1-hour)	None (free, automatic)	None for implicit; standard rate for explicit

Feature	Anthropic Claude	OpenAI	Google Gemini
Default TTL	5 min (refreshes on hit)	5–10 min; up to 24hr (extended)	60 min (explicit); dynamic (implicit)
Min cacheable tokens	1,024–4,096	1,024	1,024–4,096
Break-even	**1.4 cache reads** (5-min TTL)	Immediate (no write premium)	Immediate for implicit

Sources: Anthropic platform docs, Claude API Docs Claude API Docs OpenAI developer docs, Openai Openai Google AI/Vertex AI docs google +2 [vendor documentation].

Claude Code demonstrates 92% cache hit rate in practice

The LMCache team reverse-engineered Claude Code's behavior on a SWE-bench task (December 2025) [practitioner/community evidence], finding **92% prefix reuse across 92 LLM calls** consuming ~2M input tokens. lmcache Without caching, 2M tokens at Sonnet 4.5 rates = $6.00; with 92% cache efficiency: **$1.15 total—81% cost reduction**. Daily Dose of DS The design principle: Claude Code is "built entirely around one objective: keep the cache hot." Daily Dose of DS It never mutates the prefix to update state, uses warm-up steps to pre-populate cache, and summarizes subagent results rather than passing raw output. Daily Dose of DS

Thomson Reuters Labs achieved **59–60% cost reduction** with a cache warming strategy for document analysis (30K-token research papers, 10 questions each) [practitioner/community evidence]. However, naive parallel implementation without cache warming yielded just **4.2% cache hit rate**—essentially zero benefit. Medium

The universal design principle is static-first, dynamic-last

All three providers agree on prompt structure for cache optimization:

1. Tool/function definitions (most stable)
2. System instructions and role prompts
3. Long context and background data
4. Conversation history (prior turns are stable)
5. Current user query (fully dynamic, at the end) DigitalOcean

Critical anti-patterns include non-deterministic JSON Bento serialization (causes cache misses on logically identical content), embedding variable content in system prompts, GitHub and context compaction without cache awareness [practitioner/community evidence].

OpenAI's Responses API shows **40–80% better cache utilization** than Chat Completions because raw chain-of-thought tokens persist between turns Openai [vendor documentation]. The academic frontier is moving toward **agentic plan caching** (arXiv:2506.14852, 2025) [preprint], which extracts and reuses structured plan templates across semantically similar tasks — arXiv recognizing that existing KV-cache designed for chatbots is insufficient for agents.

Research gap: No peer-reviewed study comprehensively benchmarks cache hit rates across different workflow types or measures the interaction between context compaction strategies and cache effectiveness.

Topic 8: RUDE scoring fills a genuine gap in dependency-aware prioritization

Established frameworks lack dependency and unblocking dimensions

RICE scoring (McBride, Intercom, ~2016) [practitioner/community evidence] is the most widely adopted framework Intercom at **38% of product teams** (ProductLift survey, 2026). Formula: (Reach × Impact × Confidence) / Effort. HelloPM Known blind spots: no time sensitivity, no dependency awareness, and self-assessed confidence enables optimistic over-scoring. Fygurs fygurs **No peer-reviewed academic validation exists** for RICE effectiveness.

MoSCoW (Clegg, 1994, via DSDM) [practitioner/community evidence] provides categorical prioritization Timebee but cannot rank within categories, LaunchNotes doesn't accommodate dynamic changes, Timebee and concentrates 60%+ of effort on "Must-haves." Agile Business Consortium Voola & Babu (2024) [peer-reviewed] found the method "may not be as effective and precise as previously believed." ResearchGate

WSJF (SAFe/Leffingwell) [vendor documentation] is the closest to RUDE's multi-dimensional approach, with Cost of Delay = User-Business Value + Time Criticality + Risk Reduction/Opportunity Enablement, divided by Job Size. ProductPlan It explicitly accounts for time sensitivity—RICE's biggest gap—but does not include confidence or dependency-graph position. Fygurs fygurs

ICE scoring (Sean Ellis, growth hacking community) [practitioner/community evidence] uses Impact × Confidence × Ease, all scored 1–10. Fygurs Fast (30 items in 20 minutes) but lacks reach, cost-of-delay, and dependency dimensions. fygurs

"Unblocking" as a prioritization dimension is novel

No published framework explicitly incorporates "unblocking" or dependency graph position as a formal scoring dimension. The SPACE framework (Forsgren et al., *ACM Queue*) [peer-reviewed] explicitly mentions unblocking as a key collaborative activity improving developer productivity, and research shows constantly shifting priorities cut productivity by **up to 40%** [practitioner/community evidence]. WSJF's Risk Reduction/Opportunity Enablement partially

captures enabling future work but not explicitly. ProductPlan fygurs Critical Path Method (Kelley & Walker, 1950s) handles dependencies as scheduling constraints, not scoring criteria. This represents a **genuine novelty** of the RUDE framework.

AI-assisted prioritization is nascent

Sami et al. (2024–2025) at Tampere University [peer-reviewed] developed multi-agent LLM systems for user story prioritization using AHP, MoSCoW, and 100 Dollar Test methods. Models showed only **moderate alignment with expert rankings** per Kendall's Tau distance, with variability across runs remaining a challenge. Practitioner deployments show more promise: TechSur Solutions reported **40% of vague tickets auto-expanded** and planning meetings reduced from 4 hours to 90 minutes [practitioner/community evidence]. TechSur Solutions

The MCDA (Multi-Criteria Decision Analysis) literature provides theoretical support for multi-dimensional scoring. Hansen & Devlin (2019) [peer-reviewed] documented that structured MCDA reduces decision time by **40%** while increasing stakeholder satisfaction by **60%**. Perini et al. (2009, *Information and Software Technology*) [peer-reviewed] found that simpler methods can match complex ones (AHP) on accuracy while offering better usability— ScienceDirect relevant to RUDE's four-dimension design versus more elaborate frameworks.

The systematic literature review by Bugayenko et al. (2023, *PLOS ONE*) [peer-reviewed] documented a clear shift toward ML-based and algorithmically-assisted task prioritization, with SVM as the most widely used algorithm for task assignment. Most work focuses on bug prioritization rather than feature/task prioritization. PLOS

Research gaps: No peer-reviewed study validates any scoring framework's effectiveness in controlled settings. No framework explicitly scores dependency-graph position. The optimal number of scoring dimensions for software task prioritization has not been empirically determined. Dynamic re-prioritization (automatically recalculating scores) has no peer-reviewed validation.

Conclusion: where the evidence is strong and where gaps remain

The research landscape for WP-003's core architecture reveals a split between well-evidenced and genuinely novel territory. **Strong evidence supports** the token economics argument (context rot is universal, arXiv RAG remains 100× cheaper, pricing has fallen 1,000×), prompt caching effectiveness (60–92% savings achievable), the inadequacy of TF-IDF versus hybrid retrieval, and cross-model routing economics (50–98% cost savings). **The 85–95% diff-based compression claim is well-contextualized** by LLMLingua's 95% and Lewis's 91% measurements, though a direct controlled study specific to diff-based injection does not yet exist.

Three areas represent genuine research frontiers where ContextKeeper's contributions would be novel: (1) context exhaustion prediction—no published work applies time-series forecasting to this problem, and the Bai et al. $r < 0.15$ finding suggests linear regression is particularly ill-suited; (2) cross-model state transfer during multi-step plans—Khraishi et al.'s ±4 F1 drift is the only measurement, with no portable state format standardized; and (3) dependency-aware task scoring—no framework explicitly incorporates unblocking or graph position as scoring dimensions. These gaps position WP-003 not just as a system description but as a contribution to open problems in the field.